

Módulo 2 — Prompt Engineering Profesional

Capítulo 21 — Laboratorios de Prompt Engineering

"La ingeniería se aprende comprendiendo conceptos, pero se domina resolviendo problemas."

Objetivos de aprendizaje

- Comprender la finalidad de los laboratorios del módulo.
 - Integrar los conceptos estudiados en escenarios prácticos.
 - Introducir una metodología sistemática para experimentar con prompts.
 - Preparar al lector para resolver casos reales de AI Engineering.
-

Introducción

Los capítulos anteriores desarrollaron los fundamentos del Prompt Engineering desde una perspectiva de ingeniería. Se estudiaron patrones, prácticas para producción, ingeniería conversacional y arquitecturas basadas en prompts.

A partir de este capítulo cambia la dinámica del aprendizaje. El objetivo deja de ser incorporar nuevos conceptos y pasa a ser **aplicarlos** en situaciones representativas del mundo profesional.

Cada laboratorio propone un problema, una estrategia de resolución y un conjunto de criterios para evaluar el resultado obtenido. Los laboratorios no buscan presentar "la respuesta correcta", sino reproducir el proceso de diseño propio de un AI Engineer.

El capítulo recorre cinco laboratorios progresivos:

Laboratorio	Sección	Competencia principal
Clasificación	02	Diseñar prompts para categorización consistente.
Extracción estructurada	03	Transformar lenguaje natural en datos utilizables.
Generación controlada	04	Respetar restricciones de formato y estilo.
Ingeniería conversacional	05	Administrar estado, contexto y memoria.
Integración	06	Coordinar múltiples componentes dentro de una arquitectura.

¿Por qué laboratorios?

Leer un prompt bien diseñado permite comprender una técnica. Construirlo, evaluarlo y mejorarlo permite desarrollar una competencia.

Por este motivo, los laboratorios no buscan presentar "la respuesta correcta", sino reproducir el proceso de diseño propio de un AI Engineer.



Cada iteración aporta información para la siguiente. El laboratorio no termina cuando el prompt funciona una vez, sino cuando demuestra un comportamiento consistente ante un conjunto de pruebas variado.

Metodología de trabajo

Todos los laboratorios del módulo seguirán una estructura común.

Etapa	Objetivo
Análisis	Comprender el problema de negocio.
Diseño	Elaborar una primera versión del prompt.
Ejecución	Probar la solución con distintos casos.
Evaluación	Medir calidad, consistencia y costo.
Refinamiento	Incorporar mejoras basadas en evidencia.

Esta metodología refleja un ciclo de mejora continua similar al utilizado en proyectos reales.

Cada laboratorio debe conservar evidencia del proceso: los casos ejecutados, las versiones sucesivas del prompt, las métricas obtenidas y las conclusiones del equipo. Sin ese registro, la iteración pierde su valor metodológico.

La evaluación de cada ejecución conviene organizarla en tres dimensiones independientes:

- **Evaluación funcional:** ¿el prompt produce el resultado correcto?
- **Evaluación de formato:** ¿la salida respeta la estructura requerida?
- **Evaluación de costo y latencia:** ¿el consumo de tokens y el tiempo de respuesta son aceptables?

Separar estas dimensiones facilita el diagnóstico. Si un prompt clasifica bien pero produce un formato variable, el problema es de formato, no de razonamiento.

Buenas prácticas

Las siguientes prácticas son transversales a todos los laboratorios del capítulo:

- Formular hipótesis antes de modificar un prompt.
- Cambiar una variable por vez.
- Registrar resultados de cada iteración.
- Construir conjuntos de prueba representativos, que incluyan casos límite, ambiguos y fuera de alcance.
- Documentar las decisiones de diseño y las razones de cada cambio.
- Conservar versiones anteriores para detectar regresiones.

- Definir criterios mínimos de aceptación antes de promover una solución a entorno productivo.
-

Errores frecuentes

Los errores que aparecen con más frecuencia en laboratorios de Prompt Engineering también son transversales:

- Optimizar sin medir.
 - Cambiar varias variables simultáneamente.
 - Evaluar únicamente ejemplos favorables.
 - Concluir un laboratorio tras una única ejecución exitosa.
 - Avanzar hacia producción sin establecer umbrales mínimos de calidad.
-

Ideas clave

- Los laboratorios desarrollan criterio de ingeniería, no solo conocimiento teórico.
 - La experimentación debe ser sistemática y medible.
 - Cada iteración aporta evidencia para mejorar la solución.
 - Un laboratorio sin registro de evidencia es una demostración, no una práctica de ingeniería.
-

Transición hacia la siguiente sección

En la próxima sección comenzamos el primer laboratorio práctico: el diseño y evaluación de prompts para tareas de clasificación, aplicando la metodología presentada en esta introducción.

Módulo 2 — Prompt Engineering Profesional

Capítulo 21 — Laboratorios de Prompt Engineering

Sección 02 — Laboratorio de Clasificación

"Un laboratorio no busca demostrar que una idea funciona. Busca descubrir por qué funciona y cómo mejorarla."

Objetivos de aprendizaje

- Aplicar un proceso sistemático de diseño de prompts.
- Construir un laboratorio de clasificación de información.
- Evaluar resultados mediante criterios objetivos.
- Introducir la iteración basada en evidencia.

Introducción

La clasificación es una de las tareas más frecuentes en aplicaciones empresariales basadas en Large Language Models (LLM). Asignar una categoría a un correo electrónico, identificar el tipo de incidente de una mesa de ayuda o determinar el área responsable de un expediente son ejemplos habituales.

Aunque estas tareas parecen sencillas, representan un excelente escenario para practicar Prompt Engineering porque permiten medir con precisión el impacto de pequeños cambios en el diseño del prompt.

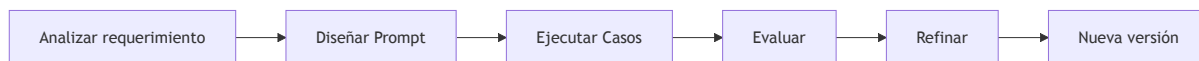
El problema

Una organización recibe cientos de consultas diarias. Cada mensaje debe clasificarse en una de las siguientes categorías:

- Recursos Humanos
- Finanzas
- Tecnología
- Compras
- Legal

El objetivo consiste en diseñar un prompt capaz de asignar correctamente cada consulta a su categoría correspondiente, de forma consistente y sin intervención manual.

Metodología



Cada iteración modifica un único aspecto del prompt para poder evaluar su impacto de forma aislada. Modificar varios elementos al mismo tiempo impide determinar cuál cambio produjo la mejora o el empeoramiento.

Conjunto de pruebas

El laboratorio debe contemplar diferentes tipos de consultas. La diversidad del conjunto de pruebas resulta tan importante como la cantidad de ejemplos.

Tipo de caso	Ejemplo
Directo	"Necesito actualizar mis datos bancarios."
Ambiguo	"Tengo un problema con un pago."
Múltiple	"No cobré el sueldo y además mi notebook no funciona."
Incompleto	"Necesito ayuda urgente."
Fuera de alcance	"¿Cómo está el clima?"

Los casos ambiguos, múltiples e incompletos son los que revelan las limitaciones más importantes del prompt. Incluirlos desde la primera iteración evita descubrirlos en producción.

Criterios de evaluación

Cada ejecución puede evaluarse mediante indicadores como:

- **Clasificación correcta:** porcentaje de mensajes asignados a la categoría adecuada sobre el total del conjunto de pruebas.
- **Consistencia entre ejecuciones:** resultado de ejecutar el mismo caso tres veces y verificar que la clasificación no varía.
- **Cumplimiento del formato solicitado:** la respuesta respeta el esquema de salida definido en el prompt.
- **Cantidad de aclaraciones requeridas:** número de casos en los que el modelo solicita información adicional en lugar de clasificar.
- **Consumo aproximado de tokens:** estimación del costo operativo de cada versión del prompt.

Estos indicadores permiten comparar distintas versiones del prompt de forma objetiva, separando la calidad funcional de la calidad de formato y del costo.

Caso de estudio

El equipo desarrolla una primera versión del prompt y obtiene buenos resultados con consultas simples. Sin embargo, las consultas que contienen más de un tema producen

clasificaciones inconsistentes: el modelo selecciona una sola categoría cuando el mensaje involucra dos áreas distintas, y esa elección varía entre ejecuciones.

En lugar de reescribir completamente el prompt, el equipo incorpora una instrucción específica para manejar múltiples intenciones: cuando el mensaje involucra más de un área, el prompt ahora indica al modelo que solicite una aclaración al usuario antes de clasificar.

La nueva versión reduce significativamente los errores sin aumentar la complejidad general. La principal enseñanza del caso es que identificar con precisión el escenario problemático — mensajes con múltiples temas— permite una intervención quirúrgica sobre el prompt, sin necesidad de alterar su estructura completa.

Buenas prácticas

- Cambiar una sola variable por iteración.
 - Registrar los resultados de todas las pruebas, incluyendo los casos fallidos.
 - Conservar versiones anteriores del prompt para detectar regresiones.
 - Incorporar nuevos casos de prueba cuando aparezcan errores en producción.
-

Errores frecuentes

- Evaluar únicamente consultas favorables.
 - Optimizar sin métricas definidas previamente.
 - Reemplazar completamente el prompt ante un fallo menor en lugar de intervenir solo sobre el aspecto problemático.
 - No documentar las decisiones tomadas en cada iteración.
-

Ideas clave

- La clasificación constituye un excelente laboratorio inicial porque permite medir resultados con precisión.
- La mejora continua depende de la evidencia obtenida durante las pruebas, no de la intuición sobre el modelo.

- Un prompt evoluciona mediante iteraciones controladas, donde cada cambio tiene una hipótesis y una medición.
-

Transición hacia la siguiente sección

En la próxima sección desarrollamos un laboratorio dedicado a la extracción estructurada de información, incorporando formatos de salida, validaciones y criterios de calidad propios de aplicaciones empresariales.

Módulo 2 — Prompt Engineering Profesional

Capítulo 21 — Laboratorios de Prompt Engineering

Sección 03 — Laboratorio de Extracción Estructurada

"La verdadera utilidad de un modelo no reside únicamente en generar texto. Reside en transformar información desestructurada en datos utilizables por otros sistemas."

Objetivos de aprendizaje

- Diseñar prompts para extracción estructurada de información.
 - Comprender la importancia de formatos de salida consistentes.
 - Aplicar criterios de validación sobre datos generados por un LLM.
 - Incorporar prácticas de ingeniería orientadas a la integración con aplicaciones empresariales.
-

Introducción

Uno de los usos más frecuentes de los Large Language Models (LLM) consiste en convertir información escrita en lenguaje natural en estructuras que puedan ser procesadas automáticamente. Correos electrónicos, documentos, contratos, expedientes o conversaciones contienen gran cantidad de información útil, pero difícil de utilizar directamente desde una aplicación.

A diferencia del laboratorio anterior, donde el objetivo era asignar una categoría, aquí el desafío consiste en extraer múltiples campos de forma precisa y devolverlos en un formato estable que otro sistema pueda consumir sin procesamiento adicional.

El problema

Una organización recibe solicitudes por correo electrónico para registrar incidentes técnicos. Cada mensaje debe transformarse en un registro con la siguiente información:

- nombre del solicitante;
- área responsable;
- prioridad;
- descripción resumida;
- fecha estimada del incidente.

La salida será utilizada directamente por un sistema de gestión de tickets, sin edición humana intermedia.

Estrategia de resolución



El éxito del laboratorio no depende únicamente de identificar la información correcta, sino también de producir un formato consistente en todas las ejecuciones. La salida del modelo requiere validación externa antes de ser consumida: incluso un prompt bien diseñado puede producir variaciones en los nombres de campos o en el orden de los datos cuando el mensaje de entrada es ambiguo o incompleto.

Casos de prueba

El conjunto de evaluación debe incluir escenarios que representen la variedad de entradas reales:

Tipo de caso	Objetivo
Información completa	Validar el comportamiento esperado.
Datos faltantes	Verificar el manejo de valores ausentes.
Información contradictoria	Evaluar las reglas de prioridad.
Mensajes extensos	Analizar la capacidad de síntesis.
Formato irregular	Comprobar robustez frente a entradas reales.

Estos escenarios permiten identificar limitaciones antes del despliegue. Los casos con datos faltantes o información contradictoria suelen ser los más reveladores: exponen cómo el prompt gestiona la incertidumbre y si genera valores inventados cuando la información no está presente.

Criterios de evaluación

La calidad del prompt puede medirse considerando:

- **Exactitud de los campos extraídos:** porcentaje de campos correctamente identificados sobre el total esperado.
- **Estabilidad del formato:** las respuestas respetan de forma consistente la estructura definida en el prompt.
- **Porcentaje de información omitida:** campos requeridos que el modelo no extrae, ya sea porque no están presentes en el mensaje o porque el prompt no los solicita con suficiente precisión.
- **Necesidad de intervención humana posterior:** número de registros que requieren corrección manual antes de ser procesados.
- **Facilidad de integración con aplicaciones consumidoras:** cantidad de transformaciones necesarias sobre la salida antes de poder utilizarla, como pasos de parseo, normalización de nombres o reordenamiento de campos.

No siempre el prompt más detallado produce el mejor resultado. La simplicidad y la consistencia suelen ser los factores determinantes para la integración con otros sistemas.

Caso de estudio

Durante las primeras pruebas, el modelo identifica correctamente los datos principales, pero utiliza distintos nombres para el mismo campo según el mensaje de entrada —"solicitante", "remitente" y "nombre del usuario" para referirse al mismo dato— y altera el orden de la salida entre ejecuciones.

El sistema de tickets espera siempre los mismos nombres de campo en el mismo orden. Cuando el prompt no los especifica, el model produce una salida semánticamente correcta pero estructuralmente inestable.

En lugar de modificar la lógica de la aplicación consumidora para adaptarse a la variabilidad del modelo, el equipo ajusta el prompt: define explícitamente los nombres exactos de cada campo y establece el orden en que deben aparecer en la salida. Como consecuencia, la complejidad del procesamiento posterior disminuye significativamente.

Este caso ilustra un principio fundamental: el Prompt Engineering no impacta solo sobre la calidad del contenido generado, sino también sobre la arquitectura del software que lo consume. Un prompt que produce una estructura estable reduce el acoplamiento entre el modelo y los sistemas integrados.

Buenas prácticas

- Definir claramente el formato esperado en el prompt, incluyendo los nombres exactos de los campos y el orden en que deben aparecer.
 - Mantener nombres de campos consistentes entre todas las versiones del prompt.
 - Validar la salida del modelo antes de consumirla, verificando que todos los campos requeridos estén presentes y con el formato correcto.
 - Diseñar prompts pensando en los sistemas que utilizarán la información: cada decisión de formato tiene consecuencias sobre la integración.
 - Especificar explícitamente cómo manejar datos ausentes: si el campo no está en el mensaje, el modelo debe indicarlo con un valor convenido (por ejemplo, `null` o `"No especificado"`).
-

Errores frecuentes

- Permitir formatos variables, delegando al modelo la decisión de cómo presentar la información.
 - Mezclar información extraída con texto explicativo o comentarios del modelo en la misma respuesta.
 - No contemplar el manejo de datos ausentes, lo que puede llevar al modelo a inventar valores.
 - Acoplar el procesamiento posterior a respuestas impredecibles, trasladando la variabilidad del prompt hacia la lógica de la aplicación.
-

Ideas clave

- La extracción estructurada es uno de los casos de uso más importantes del Prompt Engineering en aplicaciones empresariales.
 - La estabilidad del formato es tan importante como la calidad de la extracción: una respuesta correcta con formato inestable no es utilizable.
 - Diseñar pensando en la integración simplifica toda la arquitectura del sistema.
-

Transición hacia la siguiente sección

En la próxima sección desarrollamos un laboratorio centrado en generación controlada de contenido, donde el objetivo consiste en producir respuestas consistentes respetando restricciones de estilo, longitud, formato y políticas organizacionales.

Módulo 2 — Prompt Engineering Profesional

Capítulo 21 — Laboratorios de Prompt Engineering

Sección 04 — Laboratorio de Generación Controlada

"Generar texto es sencillo. Generarlo respetando reglas de negocio, formato y estilo constituye un verdadero desafío de ingeniería."

Objetivos de aprendizaje

- Diseñar prompts para generación controlada de contenido.
- Aplicar restricciones de formato, estilo y longitud.
- Comprender la importancia de la consistencia en aplicaciones empresariales.
- Evaluar la calidad de la generación mediante criterios objetivos.

Introducción

Los laboratorios anteriores abordaron tareas donde el output esperado era estructurado y verificable: una categoría, un conjunto de campos extraídos. En este laboratorio el desafío es diferente.

Muchos asistentes empresariales deben producir documentos, correos electrónicos, informes o respuestas destinadas a clientes y colaboradores. Aunque un LLM puede generar texto de alta calidad, en producción no basta con que la respuesta sea correcta. También debe cumplir restricciones previamente definidas:

- respetar un tono institucional;
- mantener un formato estable;

- limitar la longitud;
- utilizar terminología aprobada;
- evitar información no solicitada.

Este laboratorio tiene como objetivo diseñar prompts capaces de controlar esos aspectos de manera sistemática.

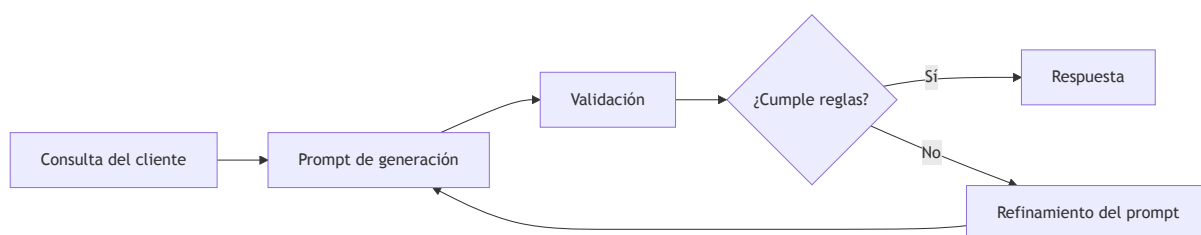
El problema

Una empresa desea automatizar la redacción de respuestas a consultas de clientes. Cada respuesta debe cumplir las siguientes reglas:

- lenguaje profesional y cordial;
- extensión máxima de 250 palabras;
- estructura fija de tres secciones;
- ausencia de opiniones personales;
- inclusión de un cierre institucional.

La salida será enviada directamente al cliente sin edición manual. Esto convierte la consistencia en un requisito tan crítico como la corrección del contenido.

Flujo del laboratorio



El ciclo continúa hasta obtener un comportamiento consistente. La validación puede realizarse mediante revisión manual con una rúbrica (¿cumple la estructura de tres secciones?, ¿respeto el límite de palabras?, ¿mantiene el tono?) o, cuando es posible, mediante verificaciones automáticas como conteo de palabras o comprobación de la presencia del cierre institucional.

Casos de prueba

Para evaluar la solución conviene utilizar consultas variadas que cubran los escenarios más exigentes:

Tipo de caso	Objetivo
Consulta simple	Validar comportamiento básico.
Reclamo complejo	Evaluar capacidad de síntesis.
Solicitud ambigua	Verificar manejo de incertidumbre.
Consulta extensa	Comprobar respeto por la longitud máxima.
Mensajes con tono agresivo	Evaluar mantenimiento del estilo institucional.

Los casos con tono agresivo y las solicitudes ambiguas son especialmente reveladores: exponen si el modelo mantiene el registro institucional bajo presión y si puede producir una respuesta válida cuando el input no es claro.

Criterios de evaluación

Los criterios de este laboratorio conviene separarlos en dos grupos:

Criterios de forma (evaluables automáticamente o con rúbrica):

- cumplimiento del formato requerido (estructura de tres secciones);
- respeto por la restricción de longitud (máximo 250 palabras);
- coherencia del estilo y tono institucional a lo largo de la respuesta;
- presencia del cierre institucional en todas las respuestas.

Criterios de contenido (requieren revisión manual):

- ausencia de información inventada o no solicitada;
- estabilidad entre distintas ejecuciones del mismo caso.

Esta separación permite transformar un criterio subjetivo ("me gusta la respuesta") en una evaluación repetible, y facilita identificar en cuál dimensión se producen los problemas cuando el prompt no rinde como se espera.

Caso de estudio

En una primera versión, el modelo genera respuestas técnicamente correctas en cuanto a contenido, pero con extensiones variables —algunas de 180 palabras, otras de 340— y sin respetar de forma consistente la estructura de tres secciones. En algunos casos el cierre institucional aparece en la segunda sección en lugar de la tercera.

El equipo decide no reescribir el prompt completo. En cambio, introduce instrucciones explícitas sobre cada restricción: define las tres secciones con sus nombres exactos, especifica el límite de palabras y agrega un ejemplo de cierre institucional dentro del prompt.

Tras ejecutar nuevamente el conjunto de pruebas, las respuestas mantienen una presentación uniforme y requieren muchas menos correcciones manuales. La mejora no proviene del modelo, sino del refinamiento del prompt y de la inclusión de restricciones explícitas y verificables.

Buenas prácticas

- Especificar claramente cada restricción de salida: formato, longitud, tono, terminología y cierre.
- Separar contenido obligatorio de contenido opcional dentro del prompt.
- Validar automáticamente los criterios de forma cuando sea posible, y de forma manual los criterios de contenido.
- Medir la consistencia entre ejecuciones además de la calidad individual de cada respuesta.

Errores frecuentes

- Confiar en que el modelo mantendrá el mismo estilo sin indicaciones explícitas.
 - Mezclar múltiples objetivos en un único prompt sin separarlos claramente.
 - Evaluar únicamente la calidad del contenido e ignorar el cumplimiento de los criterios de forma.
 - Ignorar el impacto del formato sobre los sistemas o flujos que consumirán la respuesta.
-

Ideas clave

- La generación controlada requiere tanto diseño como validación sistemática.
 - Las restricciones explícitas reducen la variabilidad y hacen el comportamiento predecible.
 - Un buen prompt facilita la integración con procesos posteriores y disminuye la necesidad de edición manual.
-

Transición hacia la siguiente sección

En la próxima sección desarrollamos un laboratorio orientado al diseño de conversaciones con estado, aplicando conceptos de memoria y contexto para construir asistentes capaces de mantener interacciones prolongadas y coherentes.

Módulo 2 — Prompt Engineering Profesional

Capítulo 21 — Laboratorios de Prompt Engineering

Sección 05 — Laboratorio de Ingeniería Conversacional

"Una conversación de calidad no se mide por la cantidad de respuestas. Se mide por la capacidad de mantener un objetivo común a lo largo del tiempo."

Objetivos de aprendizaje

- Aplicar los conceptos de Ingeniería Conversacional en un caso práctico.
- Diseñar un asistente con estado, contexto y memoria.
- Evaluar la continuidad de una conversación prolongada.
- Medir la robustez frente a interrupciones y cambios de intención.

Introducción

Los laboratorios anteriores se centraron en tareas puntuales: clasificar un mensaje, extraer campos de un correo, generar una respuesta con restricciones de formato. En cada caso, el desafío consistía en responder correctamente a una entrada única.

En este laboratorio el desafío cambia. El objetivo ya no consiste en responder correctamente un único mensaje, sino en mantener una conversación coherente durante múltiples interacciones, administrando el estado del proceso y reconstruyendo el contexto cuando resulte necesario.

Para trabajar con este laboratorio conviene precisar la diferencia entre tres conceptos que suelen confundirse:

- **Estado:** los datos estructurados que describen el progreso del proceso (ticket abierto, información ya recopilada, etapa actual del flujo).
- **Contexto:** la información que se envía al LLM en cada turno para que pueda generar una respuesta adecuada; es una selección del estado, no el estado completo.
- **Historial:** el registro cronológico de todos los intercambios de la conversación; su uso directo como contexto suele ser costoso e ineficiente en conversaciones prolongadas.

Mantener estas tres nociones separadas es la clave del diseño conversacional robusto.

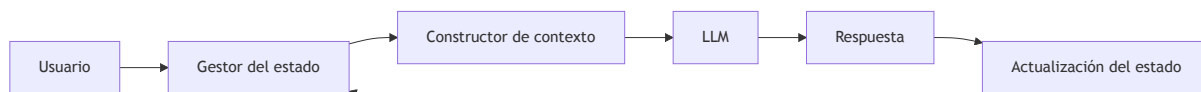
El problema

Una organización desea implementar un asistente para gestionar solicitudes de soporte interno. Durante una misma conversación el usuario puede:

- informar un incidente;
- consultar el estado de un ticket;
- corregir información previamente enviada;
- realizar preguntas adicionales;
- retomar una solicitud iniciada horas antes.

La solución debe conservar continuidad sin reenviar permanentemente todo el historial al modelo en cada turno, ya que eso incrementa el costo de inferencia y puede superar el límite de la ventana de contexto (context window) en conversaciones extensas.

Arquitectura del laboratorio



El **Gestor del estado** mantiene los datos estructurados del proceso: el incidente reportado, la información ya recopilada y la etapa actual del flujo. No es el historial completo, sino una representación compacta del progreso.

El **Constructor de contexto** selecciona, a partir del estado, qué información es relevante para el turno actual y la incluye en el prompt enviado al LLM. Dependiendo de la etapa del proceso, el contexto puede incluir el resumen del incidente, la última pregunta del usuario o el estado del ticket.

El foco del laboratorio no es el modelo en sí, sino la arquitectura que sostiene la conversación.

Casos de prueba

Escenario	Objetivo
Conversación lineal	Validar el flujo básico sin interrupciones.
Cambio de intención	Verificar recuperación del contexto tras un desvío.
Corrección de datos	Actualizar el estado sin inconsistencias.
Conversación extensa	Evaluar la administración del contexto a lo largo del tiempo.
Reanudación de sesión	Comprobar el uso de memoria persistente entre sesiones.

Las pruebas de conversación extensa y de reanudación de sesión son las más importantes: son las que revelan si la arquitectura escala más allá de los casos simples.

Criterios de evaluación

La solución puede evaluarse mediante indicadores técnicos:

- **Continuidad de la conversación:** el asistente retoma el proceso en el punto correcto tras una interrupción.
- **Consistencia del estado:** los datos almacenados en el estado son coherentes con lo que el usuario expresó a lo largo de la conversación.
- **Recuperación correcta del contexto:** la información enviada al modelo en cada turno es relevante y no contiene datos del proceso que no corresponden a ese momento.
- **Cantidad de información redundante enviada al modelo:** indicador del costo operativo de la arquitectura; una buena solución envía solo lo necesario.

- **Número de turnos necesarios para completar el proceso:** permite comparar versiones del asistente y detectar cuando el diseño conversacional genera fricciones innecesarias.
-

Caso de estudio

Durante las pruebas, un usuario inicia un ticket, interrumpe la conversación para consultar una política interna y luego retoma el incidente original. La primera implementación envía todo el historial como contexto en cada turno. Cuando el usuario retoma el incidente, el modelo pierde el hilo del proceso y solicita nuevamente información que el usuario ya había proporcionado.

El problema no está en el modelo: está en la ausencia de un estado estructurado. Sin un componente que registre qué datos ya fueron recopilados y en qué etapa se encuentra el proceso, el asistente no tiene forma de retomar desde donde se detuvo.

Tras incorporar un estado estructurado y un constructor dinámico de contexto, el asistente retoma el proceso exactamente donde había quedado. La mejora no surge del modelo, sino del diseño de la arquitectura conversacional.

Buenas prácticas

- Mantener el estado separado del historial; el estado contiene lo que importa, el historial contiene todo lo que se dijo.
 - Recuperar únicamente el contexto relevante para el turno actual, no el estado completo.
 - Registrar eventos significativos de la conversación en el estado, no como texto libre sino como datos estructurados.
 - Validar la continuidad mediante pruebas que incluyan conversaciones prolongadas y reanudaciones de sesión.
-

Errores frecuentes

- Utilizar el historial completo como única memoria: encarece la inferencia y no escala en conversaciones largas.
 - Reiniciar el flujo ante cualquier interrupción, en lugar de retomar desde el estado almacenado.
 - Mezclar estado, memoria y contexto: tratar los tres como si fueran la misma cosa dificulta el diagnóstico cuando el asistente falla.
 - No probar conversaciones extensas: los problemas de gestión de contexto solo aparecen cuando la conversación supera cierta duración.
-

Ideas clave

- La calidad conversacional de un asistente depende de la arquitectura, no solo del modelo.
 - Estado y contexto son componentes explícitos del diseño que deben definirse antes de construir el prompt.
 - Una conversación robusta debe mantener continuidad frente a cambios de intención e interrupciones.
-

Transición hacia la siguiente sección

En la próxima sección desarrollamos el laboratorio integrador, donde combinamos clasificación, extracción, generación controlada e ingeniería conversacional para resolver un caso completo de AI Engineering de principio a fin.

Módulo 2 — Prompt Engineering Profesional

Capítulo 21 — Laboratorios de Prompt Engineering

Sección 06 — Laboratorio Integrador

"La ingeniería alcanza su mayor valor cuando integra múltiples capacidades para resolver un único problema de negocio."

Objetivos de aprendizaje

- Integrar todos los conceptos estudiados durante el módulo.
- Diseñar una solución completa basada en Prompt Engineering.
- Aplicar criterios de arquitectura, evaluación y operación.
- Comprender el trabajo de un AI Engineer frente a un caso real.

Introducción

Los laboratorios anteriores abordaron capacidades específicas: clasificación, extracción estructurada, generación controlada y conversaciones de larga duración. Cada uno desarrolló una competencia en forma aislada.

En este laboratorio el desafío consiste en integrarlas dentro de una única solución. El objetivo no es construir el mejor prompt posible, sino diseñar una arquitectura capaz de resolver un problema empresarial de principio a fin.

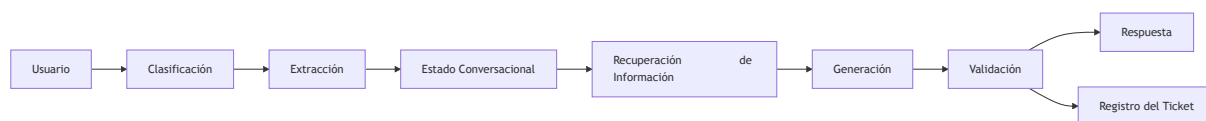
El problema

Una organización desea implementar un asistente corporativo para su mesa de ayuda. El sistema debe ser capaz de:

- interpretar la consulta del usuario;
- clasificar automáticamente el tipo de solicitud;
- extraer información relevante;
- consultar documentación interna;
- mantener una conversación coherente;
- generar una respuesta profesional;
- registrar el incidente en el sistema de tickets cuando corresponda.

El desafío consiste en coordinar todas estas capacidades sin perder mantenibilidad.

Arquitectura propuesta



Cada componente representa una responsabilidad independiente dentro de la solución. El componente de Recuperación de Información tiene como función consultar bases de conocimiento o documentación interna para enriquecer la respuesta con contenido relevante y verificado, en lugar de depender exclusivamente de lo que el modelo puede producir desde sus parámetros de entrenamiento. Si el módulo cubrió Retrieval-Augmented Generation (RAG) en capítulos anteriores, este componente es donde esa técnica se integra dentro de la arquitectura.

Plan de trabajo

El laboratorio propone recorrer las siguientes etapas.

Etapa		Objetivo
Análisis		Comprender el problema del negocio.
Diseño		Definir la arquitectura y los prompts de cada componente.
Construcción por componente		Desarrollar y probar cada componente de forma aislada.
Integración		Conectar todos los componentes y validar la interacción.
Evaluación		Ejecutar casos de prueba completos sobre el sistema integrado.
Mejora		Refinar componentes según la evidencia obtenida.

Construir y probar cada componente por separado antes de integrarlos es la práctica más importante de este laboratorio. Los errores de integración son más difíciles de diagnosticar cuando no existe una línea de base individual para cada componente.

Casos de prueba

El conjunto de evaluación debe contemplar los escenarios más representativos del sistema completo:

- consultas simples con respuesta directa;
- solicitudes incompletas que requieren aclaración;
- conversaciones largas con cambios de intención;
- casos que requieren recuperación de información desde documentación interna;
- generación y registro de tickets;
- errores de integración entre componentes;
- escenarios fuera del alcance previsto.

La solución debe demostrar estabilidad frente a todos ellos. Los casos de error de integración y fuera de alcance son los que con mayor frecuencia revelan fragilidades en la arquitectura que no se detectan en las pruebas de componentes individuales.

Criterios de evaluación

La evaluación ya no se centra únicamente en el prompt, sino en el comportamiento del sistema completo. Los criterios se distribuyen en varios niveles:

Nivel funcional:

- precisión en la clasificación de solicitudes;
- exactitud en la extracción de información;
- calidad y adecuación de la respuesta generada.

Nivel arquitectónico:

- reutilización de componentes sin duplicación de lógica;
- facilidad de mantenimiento ante cambios en uno de los componentes;
- trazabilidad de las decisiones a lo largo del flujo.

Nivel operativo:

- consistencia conversacional a lo largo de sesiones prolongadas;
- costo aproximado de inferencia por interacción;
- estabilidad de los formatos de salida de cada componente.

Caso de estudio

Una empresa implementa este laboratorio como prueba piloto. Durante las primeras iteraciones, los componentes individuales funcionan correctamente cuando se prueban por separado. Sin embargo, al integrarlos, aparecen problemas que ningún test individual había detectado: el componente de generación recibe un contexto mal formado cuando el estado conversacional no se actualiza correctamente tras una corrección del usuario; el clasificador devuelve categorías que el componente de extracción no reconoce como válidas.

El equipo identifica que el problema central es el acoplamiento entre componentes: cada uno fue diseñado asumiendo cierto formato de entrada, pero esos supuestos no estaban documentados ni probados en la integración.

La respuesta es rediseñar la arquitectura con interfaces explícitas entre componentes, desacoplar las responsabilidades y construir casos de prueba de integración específicos. Como resultado, disminuyen los errores, se reducen los costos operativos y aumenta la estabilidad del sistema.

La principal conclusión es que el éxito depende de la arquitectura mucho más que de un único prompt optimizado.

Buenas prácticas

- Diseñar primero la arquitectura, definiendo las interfaces entre componentes, y luego los prompts.
 - Validar cada componente de forma aislada antes de integrarlo.
 - Documentar los supuestos de formato de entrada y salida de cada componente.
 - Automatizar las pruebas de integración.
 - Registrar todas las versiones del sistema, no solo las versiones de los prompts individuales.
 - Medir el desempeño global del sistema además del comportamiento individual de cada componente.
-

Errores frecuentes

- Optimizar únicamente los prompts sin considerar la interacción entre componentes.
 - Ignorar los errores que aparecen solo en la integración, que no se detectan en pruebas individuales.
 - Acoplar excesivamente los componentes, haciendo que un cambio en uno rompa los demás.
 - No medir el impacto de cada cambio sobre el sistema completo.
-

Ideas clave

- Los problemas empresariales requieren integrar múltiples capacidades en una arquitectura coherente.
 - El Prompt Engineering constituye solo una parte de la solución: el diseño de las interfaces entre componentes es igualmente crítico.
 - La arquitectura determina la escalabilidad y mantenibilidad del sistema a lo largo del tiempo.
-

Transición hacia la siguiente sección

En la próxima sección realizamos el cierre del capítulo: consolidamos los aprendizajes de los cinco laboratorios y preparamos la transición hacia el Proyecto Integrador del Módulo 2.

Módulo 2 — Prompt Engineering Profesional

Capítulo 21 — Laboratorios de Prompt Engineering

Sección 07 — Cierre del capítulo y criterios de madurez

"El propósito de un laboratorio no es confirmar lo que creemos saber. Es revelar aquello que todavía necesitamos aprender."

Objetivos de aprendizaje

- Consolidar los aprendizajes obtenidos durante los laboratorios.
- Identificar criterios para evaluar la madurez de una solución basada en LLM.
- Reflexionar sobre el proceso de mejora continua.
- Preparar la transición hacia el Proyecto Integrador.

Introducción

Los laboratorios desarrollados en este capítulo reprodujeron situaciones habituales en proyectos de AI Engineering. En cada uno de ellos el objetivo fue diferente, pero el método se mantuvo constante: comprender el problema, diseñar una solución, medir resultados e iterar sobre la evidencia obtenida.

Esta forma de trabajo constituye una de las principales diferencias entre experimentar con Inteligencia Artificial (IA) y desarrollar soluciones empresariales. Experimentar con IA puede hacerse sin documentación, sin métricas y sin versiones. Desarrollar una solución empresarial requiere que cada decisión sea registrada, cada cambio sea medido y cada iteración deje evidencia para la siguiente.

Síntesis de los laboratorios

Laboratorio	Competencia principal	Técnica clave
Clasificación	Diseñar prompts para categorización consistente.	Instrucciones explícitas de categorías y manejo de múltiples intenciones.
Extracción estructurada	Transformar lenguaje natural en datos utilizables.	Formato de salida forzado con nombres de campos definidos.
Generación controlada	Respetar restricciones de formato y estilo.	Restricciones explícitas de estructura, longitud y tono.
Ingeniería conversacional	Administrar estado, contexto y memoria.	Gestión explícita de estado separado del historial.
Integración	Coordinar múltiples componentes dentro de una arquitectura.	Interfaces definidas entre componentes y pruebas de integración.

Cada laboratorio desarrolló una capacidad específica que será reutilizada en proyectos de mayor complejidad.

Criterios de madurez

Antes de considerar finalizada una solución, conviene responder las siguientes preguntas:

- ¿El comportamiento es consistente? (Referencia orientativa: al menos el 85% de los casos del conjunto de prueba producen resultados equivalentes en tres ejecuciones consecutivas.)
- ¿Puede reproducirse el resultado? ¿Existe un conjunto de pruebas documentado que permita verificarlo?
- ¿Existen métricas objetivas para cada criterio de evaluación relevante?
- ¿La arquitectura facilita el mantenimiento? ¿Un cambio en un componente puede hacerse sin romper los demás?
- ¿El costo operativo resulta aceptable para el volumen esperado?
- ¿La solución resuelve efectivamente el problema del negocio que motivó el proyecto?

Estas preguntas desplazan el foco desde el modelo hacia el valor entregado por la solución. Una demostración exitosa no responde ninguna de ellas.



Caso de estudio

Dos equipos desarrollan asistentes similares. El primero obtiene respuestas de gran calidad en demostraciones, pero carece de pruebas documentadas, versionado de prompts y métricas de desempeño. Cuando el modelo cambia de versión o el volumen de consultas aumenta, no tiene forma de saber qué empeoró ni por qué.

El segundo equipo produce resultados inicialmente más modestos, pero mantiene un proceso sistemático: cada cambio tiene una hipótesis, cada iteración tiene métricas, cada incidente se convierte en un nuevo caso de prueba. Cuando algo falla, pueden identificar el componente y la versión responsable.

Después de varios meses, el segundo equipo logra una plataforma más estable, más económica y más fácil de evolucionar. La diferencia no radica en el modelo utilizado, sino en la disciplina de ingeniería aplicada durante el desarrollo.

La deuda técnica en los prompts

Uno de los errores más frecuentes al cerrar un proyecto de Prompt Engineering es ignorar la deuda técnica acumulada en los prompts. La deuda técnica de prompts ocurre cuando una solución crece sin documentación, sin versionado y sin criterios de diseño explícitos: prompts que funcionan pero nadie sabe exactamente por qué, instrucciones duplicadas en distintos componentes, comportamientos que dependen de características no especificadas del modelo.

Esta deuda tiene consecuencias concretas: cuando el modelo actualiza su versión, cuando el volumen de casos aumenta o cuando un nuevo miembro del equipo necesita modificar el sistema, la ausencia de documentación convierte cada cambio en un riesgo. Un prompt no documentado es tan difícil de mantener como cualquier otro artefacto de software sin especificación.

El antídoto es tratar cada prompt como un artefacto de ingeniería: con versión, con pruebas, con documentación de las decisiones de diseño y con criterios de aceptación definidos.

Buenas prácticas

- Documentar cada decisión relevante, incluyendo las razones por las que se descartaron alternativas.
 - Medir antes de optimizar: sin métricas, no hay evidencia de que un cambio mejoró la solución.
 - Incorporar retroalimentación de usuarios reales en el conjunto de pruebas.
 - Revisar periódicamente la arquitectura para detectar acoplamiento innecesario.
 - Convertir cada incidente en producción en un nuevo caso de prueba documentado.
-

Errores frecuentes

- Considerar terminada una solución después de una demostración exitosa.
 - Optimizar sin evidencia: cambiar el prompt porque "parece que puede mejorar" sin una hipótesis medible.
 - Acumular deuda técnica en los prompts: crecer sin documentación, sin versiones y sin criterios de diseño explícitos.
 - No capturar el aprendizaje obtenido durante los laboratorios, lo que obliga a redescubrir los mismos errores en el siguiente proyecto.
-

Ideas clave

- La práctica desarrolla criterio de ingeniería; sin un proceso sistemático, la experiencia no se acumula.
 - La calidad de una solución depende del proceso completo, no de una única iteración exitosa.
 - Cada laboratorio constituye un paso hacia soluciones empresariales de mayor escala.
-

Transición hacia el siguiente capítulo

En el próximo capítulo desarrollamos el **Proyecto Integrador del Módulo 2**, donde todos los conceptos estudiados se aplicarán en el diseño de una solución completa de AI Engineering, siguiendo un proceso similar al de un proyecto profesional.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."